# Ensemble Model for Stock Price Prediction: Combining Multilayer perceptron (MLP) and LSTM Recurrent Neural Network (RNN)

Iva I. Ivanova (s5614260)
Katya T. Toncheva (s5460786)
Petar I. Penchev (s4683099)

**Abstract:** Stock price prediction is a challenging task due to the complex and volatile nature of financial markets. This project presents an ensemble model combining a Multilayer perceptron (MLP) and a Long Short-Term Memory (LSTM) Recurrent Neural Network to forecast closing prices. The MLP predicts residuals—the difference between stock closing prices and a trendline derived from a Simple Moving Average (SMA)—while the LSTM extrapolates the SMA to model underlying trends. Using the stock data of Apple Inc., we trained and evaluated over 5,000 MLP and 500 LSTM configurations to optimize model performance. The ensemble model achieved a Mean Absolute Error (MAE) of 2.14 USD on unseen test data. These results underscore the potential of hybrid models in financial forecasting, while also highlighting the challenges posed by the probabilistic nature of the stock market.

## 1 Introduction

This project aims to develop a robust ensemble model for predicting stock closing prices by combining two powerful neural network architectures: a feedforward neural network (FFNN) and a long short-term memory (LSTM) recurrent neural network (RNN).

The FFNN will focus on predicting residuals, which represent the difference between stock closing prices and the trendline (calculated as the simple moving average, SMA). Meanwhile, the LSTM will model the underlying trend by extrapolating the SMA. By integrating these two components, the ensemble model strives to deliver more accurate stock price predictions.

This project builds on previous work [, 2024 (S4683099)]. In the previous work, the trend was extrapolated using simple linear regression, and the MLP's predictions were overlaid on the extrapolated trend to forecast the closing prices. This report builds upon these concepts, refining the approach to combine both FFNN and LSTM for improved prediction accuracy.

We utilizes data sourced from Yahoo Finance, processed using the yfinance library, to train the models.

## 2 Theory

### 2.1 Multilayer Perceptron (MLP)

A Multilayer Perceptron (MLP) is a type of artificial neural network designed for supervised learning. It consists of an input layer, $N$ hidden layers, and the output layer. It receives raw data features through the input layer and does all its processing in the hidden layers through the weighted connections and the various non-linear activation functions. At last the output layer generates the predictions.

An MLP uses forward propagation to compute outputs and backprop to adjust weights based on the error between predictions and actual values, usually and this is the case for this paper the loss function is the Mean Square Error.

## 2.2 Long short-term memory (LSTM) Recurrent Neural Network (RNN)

Recurrent Neural Networks (RNNs) build on the MLP's structure by allowing the formation of connections that take the output from one step and use it as input for the next step. In other words, RNNs remember output information from previous steps, which makes them suitable for executing sequential tasks. RNNs have only one hidden state, which means there is only one input at each step that is calculated based on the data from the previous steps in the sequence. Therefore, RNNs are not very suitable for learning long-term dependencies in a sequential task like our goal of predicting the stock market.

A more suitable choice is the Long Short-Term Memory (LSTM) model - a Recurrent Neural Network that besides the hidden state also has a memory cell that can store information for longer periods of time. This gives the LSTMs the ability to either keep or discard information in different steps in the network. That makes LSTMs better at learning both short-term and long-term dependencies, which makes them more suitable for predicting future prices in the stock market over a longer period of time.

## 2.3 Data

The dataset used for this project comes from Yahoo Finance's API (Yahoo (2024)) via the python package (Aroussi (2024)). For the analysis, the last 500 candlesticks of the shares of Apple Inc. have been used at an interval of 1 day. The data is split up into 50 sets of 10 points. We use data from 2021-03-21 to 2024-09-01 for training and evaluation each individual model, while the dates 2024-09-01 to 2025-01-15 were used for testing the ensemble model.

### Candlesticks

A candlestick chart is a style of financial chart used to describe price movements of various assets. It is comprised of candlestick, where each candle represents four important pieces of information for that day. The open and close in the thick body, and high and low in the "candle wick". Being densely packed with information, it tends to be useful in analyzing patterns. Usually they are good indicators for the short to midterm price fluctuations.

**Figure 2.1: Candlestick chart of EUR/USD currency pair on daily timeframe. This is a chart from the MetaTrader 5 trading platform.**

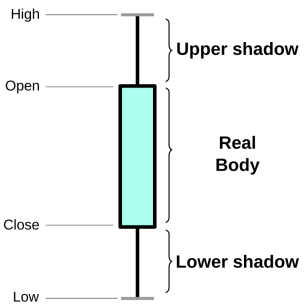


**Figure 2.2: A single candlestick.**

### Rolling Average (SMA)

A Rolling Average, or Simple Moving Average (SMA), is a statistical method used to smooth time series data by calculating the average of a fixed number of consecutive data points. Every (SMA) has a window size that determines how many points are included in each average calculation. The Rolling Average can be modeled as and FIR filter with the following difference equation,

$$y[n] = \frac{1}{N} \sum_{k=0}^{N-1} x[n-k], \tag{2.1}$$

where $N$ is the window size. For the purpose of this study we are using a window size of 3 and our difference equation becomes,

$$y[n] = \frac{1}{3} \left( x[n] + x[n-1] + x[n-2] \right). \tag{2.2}$$

This technique is a standard when it comes to technical analysis and modeling trends in stock data. Refer to the plot on the top of Figure 2.3 for a visual representation of the Simple Moving Average (SMA).

## Residuals

The main problem in trying to make any financial predictions is the influence that wildly unpredictable factors have on a given financial asset. Due to the randomness in each closing price we decided to utilize the simple moving average (SMA) described above to reduce the complexity of the problem to a movement around a trend line. The residuals tell us this movement and they are defined as the difference between the closing prices and the trend of the stock data. They can be easily computed as,

$$\text{Residual(date)} = \text{ClosingPrice(date)} - \text{SMA(date)}, \tag{2.3}$$

where ClosingPrice(date) is the closing price at a particular date and SMA(date) is the simple moving average (or the trend value) of a particular date. We can model the residuals as an FIR filter, by combining Equation 2.2 and 2.3 we get that the residuals are given by,

$$y[n] = x[n] - \frac{1}{3}\left(x[n] + x[n-1] + x[n-2]\right). \tag{2.4}$$

Refer to the plot on the bottom of Figure 2.3 for a visual representation of the residuals.

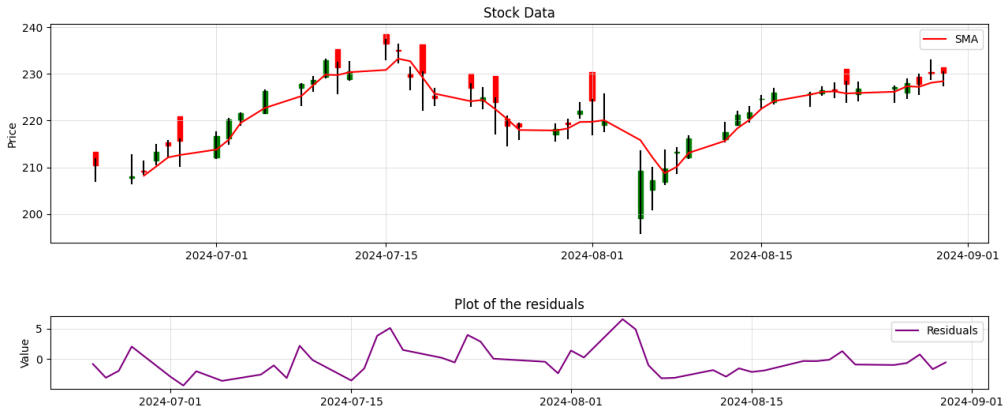


**Figure 2.3: The plot on the top represent the candlestick chart of "APPL" as taken from yahoo's API. The red line is the Rolling Average (SMA) of the closing prices. In this case the last 3 closing prices are used in computing that average. The plot on the bottom represent the residuals that are calculated using Equation 2.3.**

## Train-Test-Validation split

The train-test-validation split is a method used in machine learning to evaluate the performance of a model while providing a way of identifying overfitting or underfitting. The main idea is that it divides the available dataset into three distinct subsets which are as expected from the name, the training set, the validation set, and the test set.

The training set is as a rule of thumb the largest portion of the data and is used to train the model. The model learns patterns in the data by adjusting its parameters in order to minimize its loss function.

The validation set is used to tune hyperparameters of the model during training. By assessing the model's performance on unseen validation data, you can assess whether the model is generalizing or not. The inside that is gained from the metrics that are being monitored can help guide decisions such as learning rate adjustment, model architecture changes, or regularization.

The test set is used to evaluate the model's true generalizability. It provides an unbiased estimate of the model performance on completely unseen data.

## 2.4 Preventing overfitting

### L2 Regularization

For the residual model we used L2 regularization as it prevents overfitting by penalizing bigger weights. This happens because a new term is added to the loss function, which is the square of the weight value. In that way, it prevents the model output from depending too much on the features with larger weights, makes the values of all the feature weights smaller (but greater than zero) and therefore prevents overfitting.

### Normalization

In our project, we normalized the stock market price data for the LSTM model. This involved scaling the data values to a range between 0 and 1 using the MinMaxScaler, where the lowest value in the data is mapped to 0, the highest value to 1, and all other values are proportionally scaled as decimals between 0 and 1. Normalization ensures that every feature has an equal impact on the prediction, which is crucial for LSTMs to learn efficiently and effectively.

### Early Stoppage

One method we used to prevent overfitting is the so called *Early Stopping*. This method works by monitoring the behaviour of the loss function on the *validation* set. If the performance of the model on the validation data sees no improvements for an arbitrary number of epochs, in our case we chose 4 epochs, the training is stopped before the maximum number of epochs is reached. This is done due to the fact that if there is no improvement seen on validation data, but the model keeps improving in training, this is a clear indication that the model is memorizing the training data and failing to generalize.

## 2.5 Hyperparameter tuning

To asses if a model has generalized well, one can use the testing data set. Which as discussed earlier, is independent of both the training and the validation sets. We define the performance of the model on this data set as

$$R_{\chi}^{emp} = \frac{1}{N} \sum_{i=1}^{N} L(\mathcal{N}_{\theta_\chi}(\mathbf{u}_i), y_i), \tag{2.5}$$

where $(\mathbf{u}_i, y_i) \in T_{test}$ and $T_{test}$ is the testing data set (with $N$ elements). We use the *mean absolute error* (mae) as it is commonly the most suitable for predicting continuous outputs. With $\theta_\chi$ we denote the set of weights of the model with hyperparameters $\chi$.

In order to find the optimal set of hyperparameters ($\chi_{opt}$) that gives us the best performing model, the following minimization problem needs to be solved:

$$\chi_{opt} = \underset{\chi \in X}{\operatorname{argmin}} \; R_{\chi}^{emp}. \tag{2.6}$$

where $X$ is the search space of all the possible hyperparameters to be considered.

# 3 Results

## 3.1 MLP

### Hyperparameters

Due to the inherent randomness of the stock market, we decided to restrict ourselves to smaller MLPs, so the hyperparameter we try to explore were as follows:

- Hidden layers: 1 or 2 layers. Step size 1.
- Neurons per layer: Minimum 4, Maximum 16 (for the first layer). Minimum 0, Maximum 16 (for the second layer if it exists). Step size 2.
- Learning rate: Minimum 0.0005, Maximum 0.0100. Step size 0.0005.

- Batch Size: Minimum 1, Maximum 5. Step size 1.
- Input size: 9
- Output size: 1
- Training set percentage: 80%
- Validation set percentage: 10%
- Test set percentage: 10%

We created 5600 MLPs with different combinations of these hyperparametrs in order to do a search through the hyperparameter space in order to find $\chi_{opt}$ as described in Equation 2.6. A list of the different hyperparameter combination can be found in the Github repository.

## Initial run

Initially we decided to train the MLPs for 200 epochs to see if we actually managed to improve the training loss. We logged the validation and training loss during this training and the graph bellow shows exactly this.

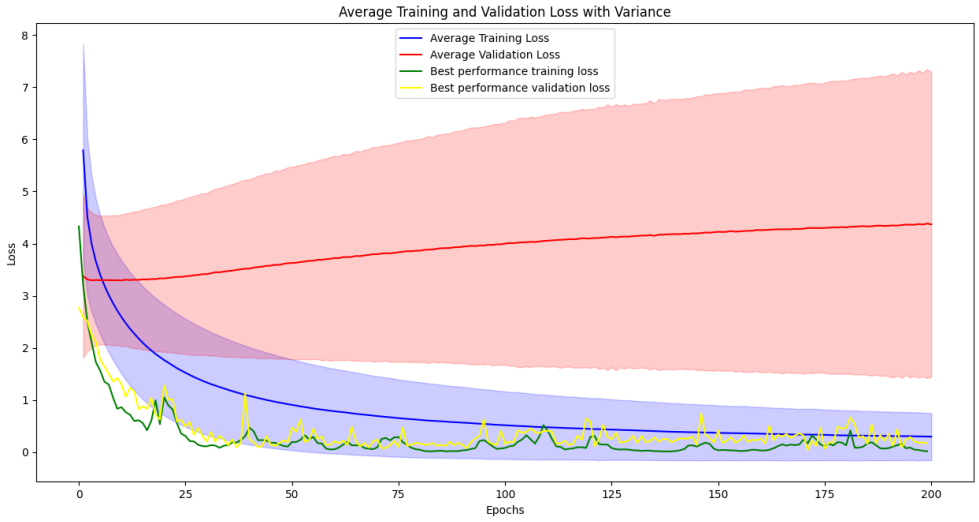


**Figure 3.1: The figure represents the average training (blue line) and validation (red line) loss for the 5,600 MLPs trained for 200 epochs. The shaded area represent one standard deviation away from the mean of the distribution (assuming a normal distribution) of the data. The green and yellow line represent the training and validation loss of the "best" performing architecture.**

We can clearly see that the objective of minimizing the training loss is achieved but the validation loss, on average, is getting worse. This is a clear sign of overfitting, or in other words, those MLPs are failing to generalize on the unseen data.

## Subsequent run involving Early Stoppage.

We made another run using the same setup as described before but to prevent overfitting we now used an *Early Stopping*. We set this early stoppage to get triggered after 4 epochs of the training process. This adds another hyperparameter to our problem but in order to save time we decided not to explore different values and kept it fixed. As a consequence of the implementation of this method the validation and training loss arrays for the different architectures are now not of the same size and we cannot create a nice plot as in Figure 3.1 for this run, however we logged the mean absolute error (mae) on the testing set to determine which architecture achieves the lowest performance, and the result of the first five MLPs can be seen in the table bellow.

| Top 5 MLPs | | | | | |
|---|---|---|---|---|---|
| **Layer 1** | **Layer 2** | **Learning Rate** | **Batch Size** | **Number of Layers** | **MAE** |
| 10 | 8 | 0.0050 | 5 | 2 | 0.214247 |
| 16 | 8 | 0.0015 | 4 | 2 | 0.254895 |
| 8 | 10 | 0.0010 | 5 | 2 | 0.346861 |
| 6 | 4 | 0.0055 | 4 | 2 | 0.437483 |
| 14 | 12 | 0.0020 | 5 | 2 | 0.481395 |

We can see that the architecture with 10 neurons in the first layer, 8 in the second, a learning rate of 0.0050, and a batch size of 5 effectively minimizes the mean absolute error (MAE) on the testing set. It is insightfull to look at the hyperparameter space that has been explored in this experiment which can be represented as a scatterplot matrix of the different parameters.

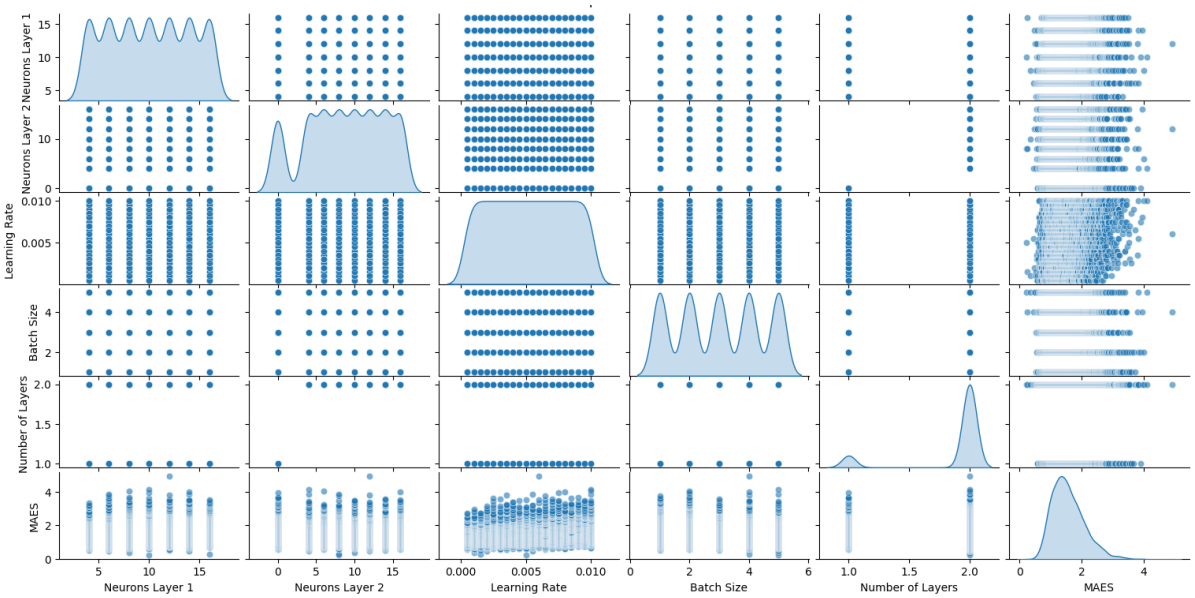


**Figure 3.2: The figure above is a scatterplot matrix which represents a collection of scatterplots, where each plot shows the relationship between two hyperparameters. The diagonal element of this matrix represents the distribution of the values of each hyperparameter.**

A summary of the parameter ranges can be seen in the table bellow. The last column of the table could be slightly misleading, as it assumes the values are IID and normally distributed.

| Parameter Ranges | | | |
|---|---|---|---|
| **Parameter** | **Max Value** | **Min Value** | **Average Value (Mean)** |
| **# Neurons Layer 1** | 16.0 | 4.0 | 10.0 |
| **# Neurons Layer 2** | 16.0 | 0.0 | 8.75 |
| **Learning Rate** | 0.01000 | 0.00050 | 0.00525 |
| **Batch Size** | 5.0 | 1.0 | 3.0 |
| **Number of Layers** | 2.0 | 1.0 | 1.875 |
| **MAES** | 4.928010 | 0.214247 | 1.568485 |

We can actually go beyond that and see how the different parameters influence the mean average error (MAE). We need to examine the correlation coefficients and the p-values. This can be seen in the table bellow.

| The correlation between the parameters and the MAE | | |
|---|---|---|
| **Parameter** | **Correlation Coefficient** | **p-value** |
| **Neurons Layer 1** | 0.165301 | 0.0 |
| **Neurons Layer 2** | -0.035256 | 0.0 |
| **Learning Rate** | 0.165337 | 0.0 |
| **Batch Size** | -0.087150 | 0.0 |
| **Number of Layers** | -0.139232 | 0.0 |

All parameters (Neurons Layer 1, Neurons Layer 2, Learning Rate, Batch Size, and Number of Layers) have statistically significant correlations with the mean absolute error (MAE) as indicated by their p-values (all of them equal to 0.0). However, one concerning fact is that the correlation coefficients are relatively small (in the range of -0.139 and 0.165), indicating very weak correlation, meaning that changes in these parameters have little to no impact on the MAE, the implications of which will be discussed in Section 4. A visual representation of the different correlation can be seen in the heatmap bellow.

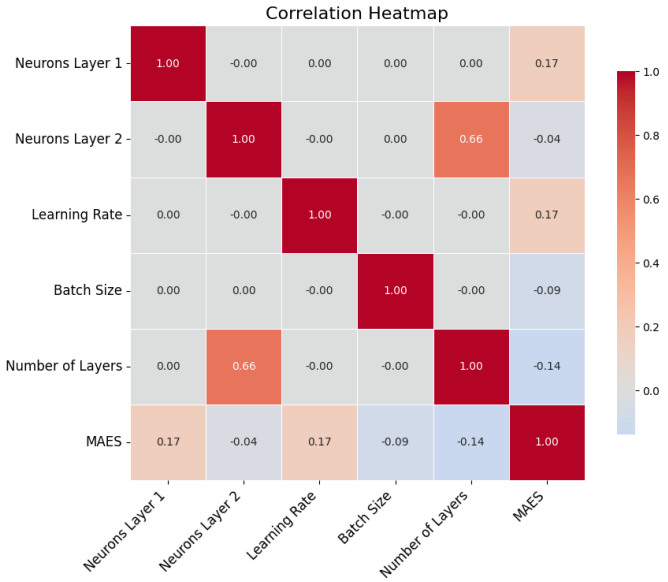


**Figure 3.3: The figure above is correlation heatmap of the relationships between different hyperparameters. It shows the strength and direction of the relationships between each pair of variables.**

## Model 145

After identifying the MLP configuration that minimized the mean absolute error (MAE) — specifically, a model with 10 and 8 neurons in its two hidden layers, respectively, a learning rate of 0.0050, and a batch size of 5 — we trained 200 identical models with the same architecture but introduced L2 regularization. We employed L2 regularization as we suspected that a deeper issue might be affecting the models' performance, particularly on the validation set.

Among these 200 runs, the model with ID 145 achieved the lowest MAE, achieving a value of 0.52 (MAE = 0.52). The loss and validation curves for this model are shown in the figure below.

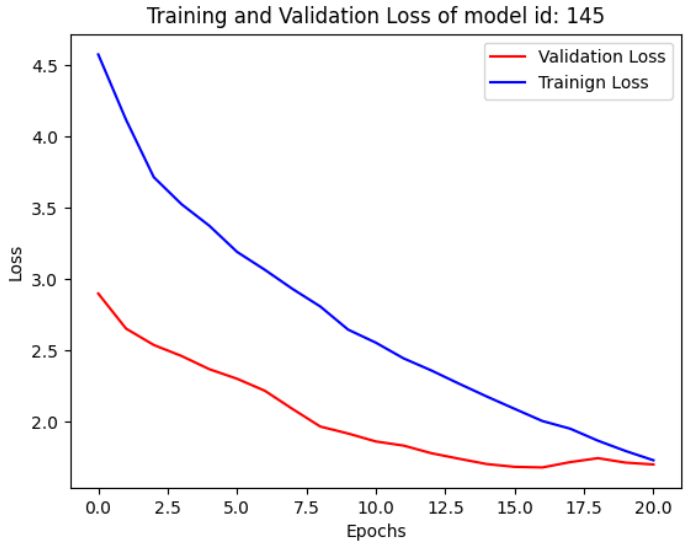


**Figure 3.4: The figure above shows the training and validation loss curves of model 145, which performed the best out of 200 runs with the top 1 architecture.**

## 3.2 LSTM

### Hyperparameters

We chose to use a simpler single-layer LSTM and hyperparameters similar to those for our MLP model.

- Hidden layers: 1 LSTM layer
- Learning rate: Minimum 0.0005, Maximum 0.0100. Step size 0.0005
- Batch Size: Minimum 1, Maximum 5. Step size 1
- Model shape: Min 9, max 81 (neurons in the LSTM layer)
- Look_back (Window Size): The number of previous data points used for predicting the next value was fixed at 9, corresponding to our input shape.
- Input shape: 9
- Output shape: 1
- Training set percentage: 80%
- Validation set percentage: 10%
- Test set percentage: 10%

To determine the optimal combination of hyperparameters, we tested 5,000 LSTM models with different values for model_shape, Learning rate, and batch size.

### Run Results

We decided to train the LSTMs for 100 epochs with the goal of improving the training loss. Fig. 3.5 shows the scaled validation and training loss during this training.

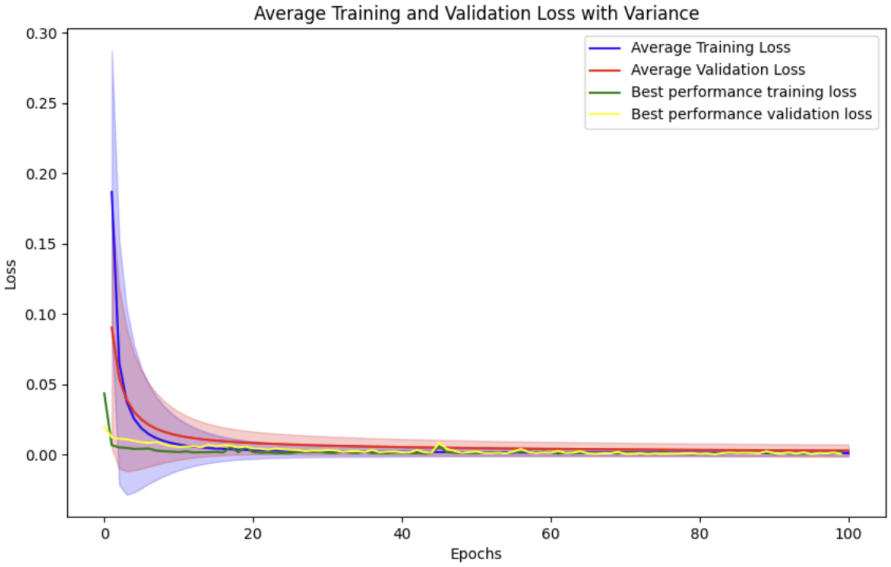


**Figure 3.5: The figure represents the average training (blue line) and validation (red line) loss for the 5,000 LSTMs trained for 100 epochs. The shaded area represent one standard deviation away from the mean of the distribution (assuming a normal distribution) of the data. The green and yellow line represent the training and validation loss of the "best" performing architecture.**

As seen in Fig. 3.5, the scaled training and validation loss both converge to 0.

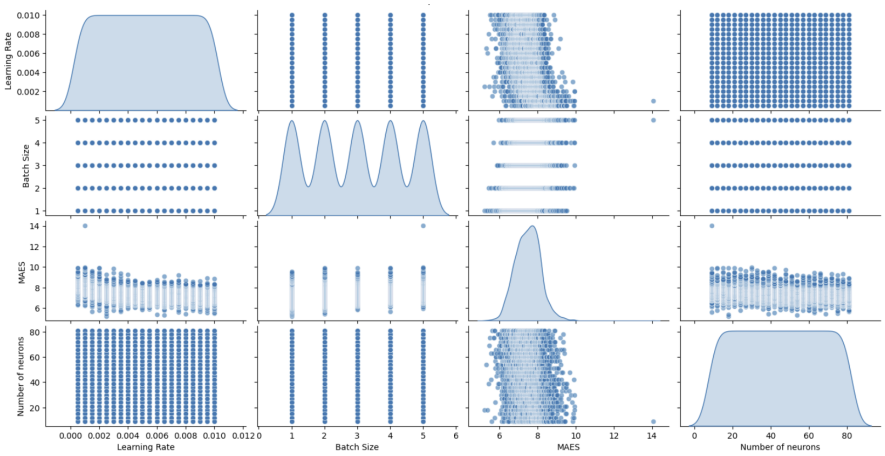


**Figure 3.6: The figure above is a scatterplot matrix and its off-diagonal elements represent the relationships between two hyperparameters of the model. Each diagonal element represents the distribution of the values of each hyperparameter.**

All parameters (Learning Rate, Batch Size, Number of Neurons) have statistically significant correlations with the mean absolute error (MAE) as indicated by their p-values (all of them equal to 0.0). However, one notable observation is that the correlation coefficients vary in magnitude, with some showing stronger relationships than others. The correlation coefficients have higher values (-0.37, 0.36) than the ones for the MLP model, which shows a stronger correlation and a more significant impact of the LSTM parameters on the stock prices' trend prediction. A visual representation of these correlations can be seen in the heatmap below.

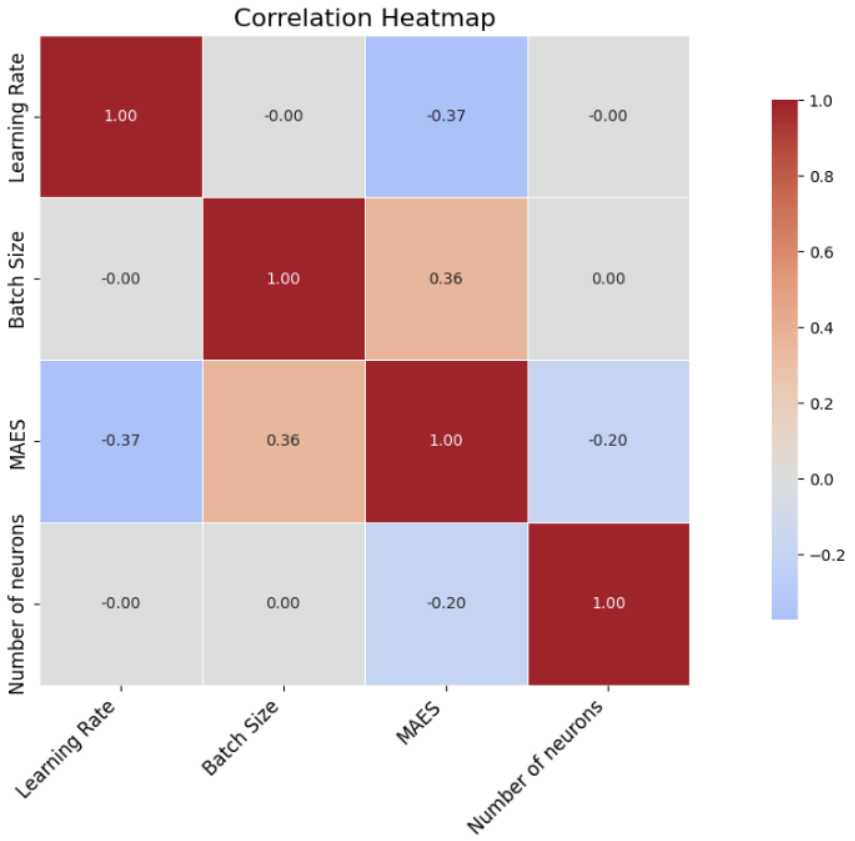


**Figure 3.7: The figure above is a correlation heatmap of the relationships between the different hyperparameters of the LSTM model. It shows the strength and direction of the relationships between each pair of variables.**

## Best Architecture

The best-performing architecture was identified based on the lowest mean absolute error (MAE) on the test set. The hyperparameters of this model are the following:

- Learning Rate: 0.0025
- Batch Size: 1
- Number of Neurons (Model Shape): 18
- MAE: 5.20

## Model 115

Using the best architecture, we trained 500 additional models. Among these, Model 115 achieved the lowest MAE and demonstrated the best generalization. The loss curves for the training and validation sets showed steady convergence with no significant overfitting. This model was ultimately integrated into the ensemble model.

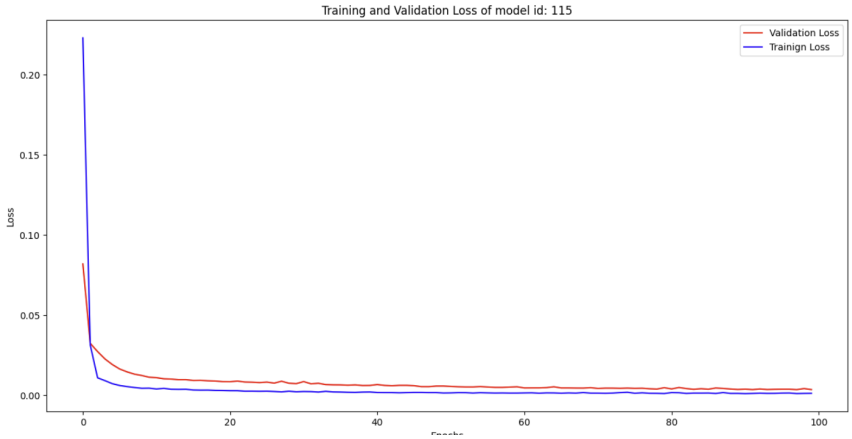


**Figure 3.8: The figure above shows the training and validation loss curves of model 115, which performed the best out of 500 runs with the top 1 architecture.**

### 3.3 Ensemble Model

The Ensemble Model combines both models from Section 3.2 and 3.1. The residual model (the MLP) predicts the next residual value, while the trend model (the LSTM) predicts the next value of the Running Average. The sum of those two gives us the next closing price, as it is clear from Equation 2.3.

We use 8 sets of 10 features to evaluate our model. We split those 8 sets into 9 input features and 1 target feature. We evaluated the predictions of the ensemble model by calculating the Mean Absolute Error (MAE) and we got a value of:

$$MAE = 2.14 \tag{3.1}$$

## 4 Discussion

Predicting stock prices is a challenging task. In this report, we implemented an ensemble of models to tackle a specific aspect of this problem. Our approach involved using an MLP (Multi-Layer Perceptron) architecture to predict the residuals of the rolling average and closing prices, while an LSTM (Long Short-Term Memory) network was used to propagate the rolling average forward.

This approach was not without its challenges. Early on, it became evident that predicting residuals is inherently difficult. We trained over 5,000 MLPs with various architectures in an attempt to identify the most effective one. However, our findings indicated that the architecture had little correlation with the model's performance, as shown by the weak correlations in Figure 3.3. This may be because small day-to-day price movements are not governed by a consistent underlying function, making them difficult for the MLP to infer. Even with the use of L2 regularization and early stopping, we were unable to improve performance. While MLPs are known as universal function approximators [Hornik et al. (1989)], a simpler statistical model might have been better suited for this task [S. Makridakis and Assimakopoulos (2018)].

The LSTM model, tasked with predicting trends, performed better. It effectively minimized both training and validation losses and achieved strong results on the test set. For this model, we experimented with a smaller variety of architectures, adjusting batch size, learning rate, and the number of neurons in the recurrent layer. After training 500 variations, we identified the architecture that minimized the Mean Absolute Error (MAE) on the test data and adopted it. Due to the strong performance on the validation set, we did not apply additional regularization, as it was deemed unnecessary.

The ensemble model, which combines the MLP and LSTM networks, performed surprisingly well on unseen test data. Evaluated over the period from September 1, 2024, to January 15, 2025, it achieved a Mean Absolute Error (MAE) of 2.14 on the closing prices.

## 5 Conclusion

In conclusion, our results highlight the complexity of stock price prediction and the need for careful model selection. While the MLP struggled to accurately predict residuals, the LSTM demonstrated its

effectiveness in capturing trends.

The ensemble approach successfully leveraged the strengths of both models, achieving strong performance on unseen data of 2.14 on the (MAE). This result also compares well with other Regression and Ensemble models for financial forecasting as it can be seen in the work of H. Runhai, where him and his team managed to get a MAE of around 1.09 across various models [He et al. (2024)]. This suggests that combining different modeling techniques can provide a more robust solution to forecasting problems.

Future work could explore further improvements, such as incorporating additional external factors and financial indicators, like bollinger bands, trading volume, long-to-short term trends etc., in training the models. Incorporating statistical techniques could also greatly improve the forcasting accuracy.

# References


Aroussi, R. (2024). yfinance. Available on GitHub. Python library. Version 0.2.44.

GeeksforGeeks (2024a). Deep Learning - Introduction to Long Short-Term Memory.

GeeksforGeeks (2024b). Introduction to Recurrent Neural Network.

He, R., Zhang, Z., Zhang, S., and Song, Q. (2024). Evaluating regression and ensemble models for financial forecasting: The case of apple stock. *ResearchGate*.

Hornik, K., Stinchcombe, M., and White, H. (1989). Multilayer feedforward networks are universal approximators. *Neural Networks*, 2(5):359–366.

Nagpal, Anuja (2024). L2 Regularization.

P. Penchev (S4683099), T. Smeman (S4950836), G. W. S. (2024). Predicting apple inc. stock using multilayer perceptron. Unpublished course project for the course 'Neural Network' [WBAI028-05] at Faculty of Science and Engineering of the University of Groningen.

S. Makridakis, E. S. and Assimakopoulos, V. (2018). Statistical and machine learning forecasting methods: Concerns and ways forward.

Yahoo (2024). Yahoo finance.

# 6 Appendix

## Codebase

All of the code used for this project, as well as the raw data collected can be found in the github repository throught the link bellow:

https://github.com/Intro-to-ML-IKP/ML-Final-Project